Artur Kis

Sensing the World

September 7, 2026

Materials Design Reflection

During Monday's lecture, we broke down how engineers actually approach programming. Specifically, we looked at the seven-step pipeline used to take a raw problem, break it into parts, and outline an algorithm before touching an IDE. Our table group spent a good chunk of class analyzing Dr. Valvano's requirements document for a TM4C123 LaunchPad security system to see how those theoretical steps apply to real hardware. That meant picking apart the specs line-by-line, mapping out 2-bit inputs for window and door contact switches, defining a 1-bit toggle to arm or disarm the system, and setting up a single LED output programmed to flash at 5 Hz. We also factored in physical constraints like hidden wiring and USB power before drawing the full flowchart logic on paper. Our diagram laid out port initializations, continuous input polling, and a decision branch running through a 100 ms wait state to handle the flashing alarm. Right after that, we put the concept into practice with a quick exercise, charting out a dual-button system that routes inputs to either a red or blue LED depending on which switch gets pressed. We wrapped up the session by jumping into GitHub, where we created a personal portfolio repository, wrote up a README in Markdown, spun up a cloud environment in Codespaces, and used Copilot to draft baseline HTML before deploying the whole person portfolio website live through GitHub.

Mapping out logic on paper before writing code saves a ton of headache down the road. When you jump straight into an editor, it is way too easy to ignore basic issues, like failing to define what the microcontroller should do if someone presses both buttons at the exact same time, or missing how a switch bounce might mess up your loop timing. Tracing through the security system diagram was definitely a bit tedious, mostly because you have to make sure the program continuously checks the armed state without getting trapped in an infinite wait block. That said, seeing how clear requirements turn directly into software flowcharts made the whole architecture side of engineering click a lot better. Building out the web portfolio was a great finish to the lab too. Getting a live site deployed before we left class felt like a real accomplishment, and it gave us a solid platform to document the rest of our work this semester.